



Ejercicios de divide y vencerás

Ej. 1 — Tienes un vector a de n enteros donde $a[1] < a[2] < \dots < a[n]$. Diseña un algoritmo de coste en tiempo en $O(\log n)$ que encuentre un índice i tal que $a[i] = i$ o devuelva que tal i no existe. Si hay varias i de este tipo, el algoritmo puede devolver cualquiera de ellas.

Ej. 2 — Diseña un algoritmo con coste en tiempo en $O(n \log n)$ que, dado un conjunto S de números reales y dado otro número real x , determine si existen o no dos elementos en S cuya suma sea exactamente x .

Ej. 3 — Diseña un algoritmo para calcular el número de unos en un vector binario (es decir, compuesto por ceros y unos), ordenado y de tamaño n , con un coste en tiempo en $O(\log n)$.

Ej. 4 — Te dan un vector v de n ($n > 1$) enteros distintos que originalmente estaba ordenado de forma creciente pero que ha sido rotado k veces, para algún valor desconocido de k entre 1 y $n - 1$. Es decir, te dan un vector $v[1, \dots, n]$ tal que el prefijo $v[1, \dots, k]$ está ordenado de forma creciente, el sufijo $v[k + 1, \dots, n]$ está ordenado de forma creciente, y $v[n] < v[1]$. Por ejemplo, este vector de 16 componentes (en este caso $k = 10$):

[9, 13, 16, 18, 19, 23, 28, 31, 37, 42, -4, 0, 2, 5, 7, 8]

Escribe un algoritmo que, dado el vector rotado, devuelva el valor k en la forma más rápida posible. Analiza su coste en tiempo.

Ej. 5 — El país mágico de Algoritmia celebró recientemente elecciones. Todos los votos emitidos fueron electrónicos, pero desafortunadamente, un aumento de energía repentino provocó un mal funcionamiento en el sistema justo antes de que se contaran los votos. De hecho, el sistema se volvió loco y solo conocemos los siguientes datos sobre la elección:

- Hay exactamente n ciudadanos en Algoritmia, y todos votaron.
- Exactamente un candidato recibió el voto mayoritario. En otras palabras, algún candidato recibió estrictamente más de $n/2$ votos.
- No sabemos cuántos candidatos hubo.

El gobierno de Algoritmia ha oído hablar de tu habilidades algorítmicas desde que has empezado el curso de Algoritmia básica. Así que te contrataron para contar los votos. Sin embargo, debido al mal funcionamiento, tienes recursos limitados. El software de ciberseguridad de Algoritmia te permite abrir solo una papeleta. Como resultado, debes encontrar una papeleta con el voto para el candidato ganador. La única función a la que tienes acceso ilimitado es:

`esCoincidente(papeletaA, papeletaB)`

Esta función devuelve *Verdadero* si `papeletaA` y `papeletaB` tienen un voto para el mismo candidato, o *Falso* en caso contrario. Diseña un algoritmo determinista, de tipo divide y vencerás, que use $O(n \log n)$ llamadas a `esCoincidente` y devuelva al candidato ganador.

Ej. 6 — Te dan una imagen de una cueva que tiene n píxeles de altura y m píxeles de anchura. Cada píxel es un 0 o un 1. Cada columna de la imagen pertenece exactamente a una de las siguientes categorías:

- Una columna es una columna mineral si cada píxel en ella es un 1.
- Una columna es una estalactita si tiene 0 y 1, y todos los unos aparecen encima de todos los 0.
- Una columna es una estalagmita si tiene 0 y 1, y todos los unos aparecen debajo de todos los 0.
- Una columna no es nada si cada píxel en ella es un 0.

La longitud de una estalactita o estalagmita es el número de 1 en esa columna. Dada tal imagen, diseña un algoritmo de coste en tiempo en $O(m \log n)$ que genere el siguiente valor:

max (longitud de la estalactita más larga, longitud de la estalagmita más larga)

Ej. 7 — Dados dos vectores de enteros $X[1, \dots, n]$ e $Y[1, \dots, n]$, ordenados de forma creciente, diseña un algoritmo que calcula la mediana de los $2n$ elementos en tiempo en $O(\log n)$ y sin utilizar vectores adicionales.

Ej. 8 — Una matriz de rastreo es una matriz cuadrada de $n \times n$ números enteros (sin repeticiones), siendo n una potencia de 2, y en la que se cumple que $a_1 < b_1 < a_2 < b_2 < a_3 < b_3 < a_4 < b_4$ y donde además las cuatro sub-matrices componentes son también matrices de rastreo. La figura a la izquierda muestra el caso general, la figura a la derecha es un ejemplo de matriz de rastreo 4×4 .

	$\leftarrow n/2 \rightarrow$							
↑	a_1				a_2			
$n/2$								
↓				b_1				b_2
	a_3				a_4			
			b_3					b_4

1	2	10	14
5	8	15	19
20	25	55	70
40	43	71	84

Diseña un algoritmo de tipo divide y vencerás que determine si la matriz es o no de rastreo. Analiza su coste en tiempo.

Ej. 9 — Has encontrado una colonia de extraterrestres que viven en Marte y has estado aprendiendo más sobre su aritmética. En particular, has descubierto que usan el mismo sistema numérico en base 10, suma y resta igual que nosotros, pero su sistema de multiplicación es completamente diferente. El símbolo que usan es $\star$ y tienen una tabla de multiplicar de 1 dígito que usan en la escuela primaria. Aquí hay unos ejemplos:

$1 \star 1 = 3$ $2 \star 3 = 19$ $3 \star 5 = 49$ $9 \star 9 = 243$

Después de un rato, te has enterado que su multiplicación corresponde a lo siguiente en la aritmética humana:

$$x \star y = x^2 + xy + y^2.$$

Quieres explicarles una forma eficiente de calcular el producto marciano de dos número de n dígitos, sin hacer referencia ni explicar ninguna multiplicación humana (incluso implícitamente como una suma). Por ejemplo, no puedes decirles que calculen x^2 , xy y y^2 por separado, ya que no entienden que significan estos términos. Encuentra un algoritmo para calcular el producto marciano de dos números de n dígito que se ejecuta en tiempo en $O(n^{\log 3})$. Por ejemplo, tu algoritmo podría tener como entrada: $x = 1234$ y $y = 4321$, y debería devolver $x \star y = 25525911$. Puedes suponer que n es una potencia de 2, y que los ordenadores de los marcianos pueden hacer operaciones de *shift*, suma y resta exactamente como los ordenadores humanos. También puedes suponer que los ordenadores de los marcianos tienen fácil acceso a la tabla de multiplicar $\star$ que has recibido.

Resolución: Búsqueda de un Punto Fijo en un Vector Ordenado

Este documento presenta la resolución simplificada para encontrar un índice i tal que $a[i] = i$ en un vector estrictamente creciente de n enteros usando la estrategia de **Divide y Vencerás**.

a) Criterios de Descarte

Al evaluar el punto medio mid de nuestro rango actual $[izq, der]$:

1. Si $a[mid] = mid$: Hemos encontrado el punto fijo. Lo devolvemos inmediatamente.
2. Si $a[mid] > mid$: El punto fijo (si existe) solo puede estar en la mitad izquierda.
 - **Decisión:** Descartamos la derecha y buscamos en $[izq, mid - 1]$.
3. Si $a[mid] < mid$: El punto fijo (si existe) solo puede estar en la mitad derecha.
 - **Decisión:** Descartamos la izquierda y buscamos en $[mid + 1, der]$.

b) Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo utilizando índices base 1:

```
Funcion BuscarPuntoFijo(a, izq, der)
Inicio
    // Caso Base: El rango se ha cruzado (no existe el punto fijo)
    Si izq > der Entonces
        Retornar -1
    FinSi

    // Calcular el punto medio de forma simple
    mid <- ParteEntera((izq + der) / 2)

    // Evaluar y descartar mitades
    Si a[mid] = mid Entonces
        Retornar mid
    Sino Si a[mid] > mid Entonces
        Retornar BuscarPuntoFijo(a, izq, mid - 1)
    Sino
        Retornar BuscarPuntoFijo(a, mid + 1, der)
    FinSi
FinFuncion
```

d) Análisis de Coste

- **Complejidad Temporal:** $O(\log n)$, ya que dividimos el espacio de búsqueda a la mitad en cada paso recursivo.
- **Complejidad Espacial:** $O(\log n)$ debido al espacio ocupado en la pila de llamadas por la recursión.

Resolución: Buscar dos elementos en S que sumen X

Este documento presenta la resolución detallada para determinar si existen dos elementos dentro de un conjunto S de n números reales que sumen exactamente un valor objetivo x , garantizando un coste temporal estricto de $O(n \log n)$.

a) Idea del Algoritmo

Para resolver este problema en $O(n \log n)$, utilizamos una estrategia combinada de **Ordenación** seguida de una búsqueda con **Dos Punteros** (también conocida como búsqueda bidireccional):

1. **Ordenación preliminar:** Primero, ordenamos el conjunto S de menor a mayor. Utilizando un algoritmo de ordenación eficiente (como *MergeSort* o *QuickSort*), este paso consume un tiempo de $O(n \log n)$.
2. **Estrategia de dos punteros:** Una vez ordenado el vector, colocamos dos punteros (índices): uno al principio del vector ($izq = 1$) y otro al final ($der = n$). Sumamos los valores a los que apuntan:
 - **Si la suma es igual a x :** ¡Hemos terminado! Hemos encontrado la pareja y devolvemos `Verdadero`.
 - **Si la suma es mayor que x :** Dado que el vector está ordenado, para reducir la suma debemos apuntar a un número más pequeño. Por lo tanto, decrementamos el puntero derecho ($der \leftarrow der - 1$).
 - **Si la suma es menor que x :** Para incrementar la suma, necesitamos apuntar a un número mayor. Por tanto, incrementamos el puntero izquierdo ($izq \leftarrow izq + 1$).

Este proceso se repite hasta encontrar los elementos o hasta que los dos punteros se crucen, garantizando que exploramos todo el espacio de soluciones de forma eficiente.

b) Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo utilizando índices con base 1:

```

Funcion ExisteSumaX(S, n, x)
Inicio
  // Paso 1: Ordenar el conjunto S de menor a mayor
  S_ordenado <- OrdenarMergeSort(S) // Coste:  $O(n \log n)$ 

  // Paso 2: Inicializar punteros en los extremos
  izq <- 1
  der <- n

  // Paso 3: Búsqueda bidireccional
  Mientras izq < der Hacer
    suma_actual <- S_ordenado[izq] + S_ordenado[der]

    Si suma_actual = x Entonces
      Retornar Verdadero // Se encontró la pareja

```

```
Sino Si suma_actual > x Entonces
    der <- der - 1 // Reducir el valor de la suma
Sino
    izq <- izq + 1 // Aumentar el valor de la suma
FinSi
FinMientras

// Si los punteros se cruzan sin éxito, no existen tales elementos
Retornar Falso
FinFuncion
```

d) Análisis de Coste

- **Complejidad Temporal:** $O(n \log n)$.
 - El paso de ordenación toma $O(n \log n)$ en el peor de los casos.
 - El bucle `Mientras` recorre el vector utilizando dos punteros que avanzan el uno hacia el otro, por lo que realiza a lo sumo n iteraciones de coste constante $O(1)$. Esto da un coste de búsqueda de $O(n)$.
 - Por tanto, el coste total es $O(n \log n) + O(n) = O(n \log n)$.
- **Complejidad Espacial:** $O(n)$ si utilizamos un algoritmo de ordenación como *MergeSort* que requiere espacio auxiliar, o bien $O(1)$ adicional (en el sitio) si usamos variantes in-place eficientes.

Resolución: Contar Unos en un Vector Binario Ordenado

Este documento presenta el diseño de un algoritmo de **Divide y Vencerás** para contar el número de unos en un vector binario ($a[i] \in \{0, 1\}$) ordenado de tamaño n en tiempo logarítmico $O(\log n)$.

a) Idea del Algoritmo

Dado que el vector a está ordenado y solo contiene ceros y unos, siempre tendrá la siguiente forma:

$$a = [0, 0, \dots, 0, 1, 1, \dots, 1]$$

El problema de contar cuántos unos hay en el vector se reduce a **encontrar el índice de la primera aparición del número 1**.

Si logramos encontrar el primer índice p tal que $a[p] = 1$ (utilizando índices con base 1):

- Todos los elementos desde la posición p hasta el final n serán obligatoriamente unos.
- La cantidad total de unos será simplemente:

$$\text{Cantidad de unos} = n - p + 1$$

- Si no existe ningún 1 en el vector, el resultado será 0.

Para encontrar esta primera transición de 0 a 1 en tiempo $O(\log n)$, aplicamos una variante de la **Búsqueda Binaria** (Divide y Vencerás).

b) Criterios de Descarte

Al evaluar la posición media mid del rango actual de búsqueda $[izq, der]$:

1. Si $a[mid] = 1$: El primer uno de la secuencia puede ser el propio mid o puede estar más a la izquierda.
 - **Decisión:** Comprobamos si es el primer elemento del vector ($mid = 1$) o si el elemento anterior es un cero ($a[mid - 1] = 0$). Si se cumple alguna de estas, hemos encontrado el primer uno y devolvemos mid . En caso contrario, descartamos la mitad derecha y seguimos buscando a la izquierda en $[izq, mid - 1]$.
2. Si $a[mid] = 0$: Dado que el vector está ordenado, todos los elementos a la izquierda de mid también son obligatoriamente ceros. El primer uno tiene que estar estrictamente en la mitad derecha.
 - **Decisión:** Descartamos la mitad izquierda y seguimos buscando en $[mid + 1, der]$.

c) Algoritmo en Pseudocódigo

A continuación se detalla el pseudocódigo recursivo que busca el índice del primer uno y calcula el total utilizando índices base 1:

```

Funcion BuscarPrimerUno(a, izq, der)
Inicio
    // Caso Base: El rango se ha cruzado y no se ha encontrado ningún 1
    Si izq > der Entonces
        Retornar -1
    FinSi

    // Calcular el punto medio
    mid <- ParteEntera((izq + der) / 2)

    Si a[mid] = 1 Entonces
        // Comprobar si es el primer 1 del vector completo
        Si mid = 1 o a[mid - 1] = 0 Entonces
            Retornar mid
        Sino
            // Seguir buscando el origen de los unos a la izquierda
            Retornar BuscarPrimerUno(a, izq, mid - 1)
        FinSi
    Sino
        // Es un 0, el primer uno tiene que estar a la derecha
        Retornar BuscarPrimerUno(a, mid + 1, der)
    FinSi
FinFuncion

Funcion ContarUnos(a, n)
Inicio
    primer_uno <- BuscarPrimerUno(a, 1, n)

    Si primer_uno = -1 Entonces
        Retornar 0 // No hay unos en el vector
    Sino
        Retornar n - primer_uno + 1
    FinSi
FinFuncion

```

d) Análisis de Coste

- **Complejidad Temporal:** $O(\log n)$. En cada paso de la recursión dividimos el espacio de búsqueda exactamente por la mitad, realizando comparaciones elementales de tiempo constante $O(1)$.
- **Complejidad Espacial:** $O(\log n)$ debido al uso de la memoria en la pila de llamadas de la recursión, la cual tiene una profundidad máxima equivalente a la altura del árbol de división recursiva ($\log_2 n$).

Resolución: Encontrar el Índice de Rotación k en un Vector

Este documento presenta el diseño de un algoritmo de Divide y Vencerás para encontrar el índice de rotación k en un vector $v[1 \dots n]$ de enteros distintos en tiempo logarítmico $O(\log n)$.

a) Idea del Algoritmo

El enunciado nos indica que el vector original estaba ordenado de forma creciente pero ha sido rotado k veces ($1 \leq k \leq n - 1$). Esto significa que el vector se divide en dos tramos crecientes bien definidos:

1. El primer tramo: $v[1 \dots k]$ (valores altos, empezando en $v[1]$ hasta el máximo $v[k]$).
2. El segundo tramo: $v[k + 1 \dots n]$ (valores bajos, empezando en el mínimo $v[k + 1]$ hasta $v[n]$).

Como $v[n] < v[1]$, el punto de inflexión donde se "rompe" el orden creciente ocurre exactamente en la frontera entre $v[k]$ y $v[k + 1]$, cumpliéndose que:

$$v[k] > v[k + 1]$$

Para encontrar este índice k en tiempo $O(\log n)$, aplicamos una búsqueda binaria. En cada paso, analizamos el elemento central $v[mid]$ para determinar de qué lado de la inflexión nos encontramos y poder descartar la mitad del vector.

b) Criterios de Descarte

Al evaluar la posición media mid del rango actual de búsqueda $[izq, der]$:

1. **Comprobación directa de éxito:**
 - Si $v[mid] > v[mid + 1]$, entonces mid es exactamente el punto de ruptura k . ¡Lo devolvemos!
 - Si $v[mid] < v[mid - 1]$, entonces el elemento anterior es el punto de ruptura ($k = mid - 1$). ¡Lo devolvemos!
2. **Decisión de descarte:**
 - Si $v[mid] > v[1]$: Esto significa que mid todavía se encuentra en el primer tramo creciente (el tramo de valores grandes). Por lo tanto, el punto de ruptura k tiene que estar obligatoriamente a su derecha.
 - **Decisión:** Descartamos la mitad izquierda y buscamos en $[mid + 1, der]$.
 - Si $v[mid] < v[1]$: Esto significa que mid ya ha cruzado al segundo tramo (el tramo de valores pequeños). Por lo tanto, el punto de ruptura k está a su izquierda.
 - **Decisión:** Descartamos la mitad derecha y buscamos en $[izq, mid - 1]$.

c) Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo recursivo utilizando índices base 1:

```

Funcion BuscarInflexion(v, izq, der)
Inicio
    // Caso Base: Si el rango se cruza o se reduce a un elemento, no hay rotación
    Si izq >= der Entonces
        Retornar -1
    FinSi

    // Calcular el punto medio
    mid <- ParteEntera((izq + der) / 2)

    // 1. Comprobaciones de éxito inmediato
    Si v[mid] > v[mid + 1] Entonces
        Retornar mid
    FinSi

    Si v[mid] < v[mid - 1] Entonces
        Retornar mid - 1
    FinSi

    // 2. Descarte de mitades comparando con el extremo izquierdo original v[1]
    Si v[mid] > v[1] Entonces
        // El punto de ruptura está a la derecha
        Retornar BuscarInflexion(v, mid + 1, der)
    Sino
        // El punto de ruptura está a la izquierda
        Retornar BuscarInflexion(v, izq, mid - 1)
    FinSi
FinFuncion

Funcion ObtenerK(v, n)
Inicio
    // Llamada inicial para buscar el punto de inflexión en todo el vector
    Retornar BuscarInflexion(v, 1, n)
FinFuncion

```

d) Análisis de Coste

- **Complejidad Temporal:** $O(\log n)$, ya que el tamaño del intervalo de búsqueda se reduce exactamente a la mitad en cada paso recursivo mediante comparaciones elementales de tiempo constante $O(1)$.
- **Complejidad Espacial:** $O(\log n)$ debido al espacio reservado en la pila de llamadas de la recursión, cuya profundidad máxima es proporcional a la altura del árbol de división recursiva ($\log_2 n$).

Resolución: Encontrar el Voto Mayoritario en Algoritmia

Este documento presenta el diseño de un algoritmo de **Divide y Vencerás** (inspirado en el esquema de *MergeSort*) para identificar la papeleta del candidato ganador (el que tiene estrictamente más de $n/2$ votos) en un censo de n ciudadanos, utilizando únicamente la función de comparación elemental `esCoincidente` con un coste de $O(n \log n)$ llamadas.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Disponemos de un conjunto de n papeletas electrónicas representadas como un vector $V = [p_1, p_2, \dots, p_n]$. No conocemos la identidad de los candidatos ni cuántos hay en total. La solución esperada es el índice i ($1 \leq i \leq n$) de cualquier papeleta que pertenezca al candidato ganador.

2. Restricciones Explícitas

La única forma de inspeccionar e interactuar con las papeletas es mediante la función proporcionada:

$$\text{esCoincidente}(p_A, p_B) \in \{\text{Verdadero}, \text{Falso}\}$$

Esta función devuelve `Verdadero` si ambas papeletas contienen un voto para el mismo candidato, o `Falso` en caso contrario.

3. Restricciones Implícitas

Garantizamos por el enunciado que existe exactamente un candidato mayoritario que ha recibido estrictamente más de $n/2$ votos en el vector total.

b) Idea y Diseño del Algoritmo (Estrategia "Merge")

Para resolver este problema en tiempo $O(n \log n)$, aplicamos una estrategia de **Divide y Vencerás**:

- División (Divide):** Dividimos el vector actual de papeletas $V[\text{izq} \dots \text{der}]$ por la mitad en dos subvectores: el izquierdo $V[\text{izq} \dots \text{mid}]$ y el derecho $V[\text{mid} + 1 \dots \text{der}]$.
- Conquista (Conquer):** Buscamos recursivamente el candidato mayoritario en cada una de las dos mitades. Sea C_{izq} el candidato mayoritario de la mitad izquierda y C_{der} el de la mitad derecha. (Si una mitad no tiene un candidato con mayoría absoluta interna, devolverá -1).
- Combinación (Merge/Combine):** Esta es la parte clave. Una papeleta solo puede ser la mayoría absoluta de un bloque grande si es la mayoría absoluta de, al menos, uno de sus dos subbloques constituyentes. Por tanto, tenemos tres escenarios de combinación:

- Si $C_{izq} = C_{der}$: Entonces ese candidato es el claro ganador mayoritario del bloque combinado actual.
- Si $C_{izq} \neq C_{der}$ (o si uno de ellos es -1): Debemos verificar cuál de los dos candidatos $(C_{izq}$ o $C_{der})$ es el verdadero ganador mayoritario del intervalo combinado actual $V[izq \dots der]$.
- **Verificación:** Contamos individualmente cuántas veces aparece C_{izq} y/o C_{der} en todo el segmento actual $V[izq \dots der]$ usando la función `esCoincidente`. Si alguno de los dos supera el umbral de la mitad del tamaño actual del segmento, se convierte en el ganador de dicho segmento. En caso contrario, el segmento actual no tiene mayoría y devuelve -1 .

c) Algoritmo en Pseudocódigo

// Función que cuenta cuántas veces coincide una papeleta candidata en un rango c
 Funcion ContarVotos(V, izq, der, papeleta_candidata)

Inicio

 contador <- 0

 Para i <- izq Hasta der Con Paso 1 Hacer

 Si esCoincidente(V[i], papeleta_candidata) Entonces

 contador <- contador + 1

 FinSi

 FinPara

 Retornar contador

FinFuncion

// Función recursiva de Divide y Vencerás

Funcion EncontrarGanadorRecursivo(V, izq, der)

Inicio

 // Caso Base 1: El subproblema se ha reducido a una sola papeleta

 Si izq = der Entonces

 Retornar V[izq] // Una única papeleta siempre es mayoría absoluta de sí misma

 FinSi

 // Calcular el punto medio para la división

 mid <- ParteEntera((izq + der) / 2)

 // Conquistar: Resolver de forma recursiva ambas mitades

 c_izq <- EncontrarGanadorRecursivo(V, izq, mid)

 c_der <- EncontrarGanadorRecursivo(V, mid + 1, der)

 // Combinar (Merge):

 // Si ambas mitades coinciden en el mismo candidato

 Si c_izq != -1 y c_der != -1 y esCoincidente(c_izq, c_der) Entonces

 Retornar c_izq

 FinSi

 // Si difieren, verificar candidatos en todo el rango actual [izq, der]

 longitud_actual <- der - izq + 1

 mitad_actual <- longitud_actual / 2

 Si c_izq != -1 Entonces

 votos_izq <- ContarVotos(V, izq, der, c_izq)

 Si votos_izq > mitad_actual Entonces

 Retornar c_izq

```

        FinSi
    FinSi

    Si c_der != -1 Entonces
        votos_der <- ContarVotos(V, izq, der, c_der)
        Si votos_der > mitad_actual Entonces
            Retornar c_der
        FinSi
    FinSi

    // Si ninguno obtiene mayoría absoluta en este segmento combinado
    Retornar -1
FinFuncion

Funcion ObtenerGanadorElecciones(V, n)
Inicio
    ganador <- EncontrarGanadorRecursivo(V, 1, n)
    Retornar ganador
FinFuncion

```

d) Análisis de Coste

Complejidad Temporal (Llamadas a `esCoincidente`)

La cantidad de llamadas a la función `esCoincidente` en un vector de tamaño m se define mediante la siguiente relación de recurrencia para el caso peor:

$$T(m) = 2T\left(\frac{m}{2}\right) + f(m)$$

Donde $f(m)$ es el número de llamadas a `esCoincidente` realizadas durante la etapa de combinación (el "Merge") para un subproblema de tamaño m :

1. Comprobar si c_{izq} y c_{der} coinciden requiere 1 llamada a `esCoincidente`.
2. Contar las ocurrencias de c_{izq} en el intervalo de tamaño m requiere m llamadas.
3. Contar las ocurrencias de c_{der} en el intervalo de tamaño m requiere m llamadas.

Por lo tanto, la función de combinación tiene un coste de $f(m) \leq 2m + 1$ llamadas. Esto nos da la relación de recurrencia exacta:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) \in O(n \log n)$$

Complejidad Espacial

La profundidad de la pila recursiva es de $O(\log n)$ debido a la bisección del espacio de búsqueda en cada paso. Puesto que no se realizan copias de los subvectores de datos (se trabaja directamente por índices sobre el vector original V).

Ejercicio 6: Análisis de Cueva (Divide y Vencerás)

Enunciado

Te dan una imagen de una cueva que tiene n píxeles de altura (filas) y m píxeles de anchura (columnas). Cada píxel es un 0 o un 1. Cada columna de la imagen pertenece exactamente a una de las siguientes categorías:

- **Columna mineral:** Cada píxel en ella es un 1.
- **Estalactita:** Tiene 0 y 1, y todos los unos aparecen encima de todos los 0.
- **Estalagmita:** Tiene 0 y 1, y todos los unos aparecen debajo de todos los 0.
- **Nada:** Cada píxel en ella es un 0.

La longitud de una estalactita o estalagmita es el número de 1 en esa columna. Dada tal imagen, diseña un algoritmo de coste en tiempo en $\mathcal{O}(m \log n)$ que genere el siguiente valor:

$\max(\text{longitud de la estalactita más larga, longitud de la estalagmita más la}$

1. Estrategia de Divide y Vencerás

El problema nos pide procesar una matriz de tamaño $n \times m$. Dado que la complejidad objetivo es $\mathcal{O}(m \log n)$, disponemos de un tiempo de $\mathcal{O}(\log n)$ **para analizar cada columna**.

Identificación de la Categoría en $\mathcal{O}(1)$

Podemos determinar con total certeza a qué categoría pertenece cualquier columna j examinando únicamente su primer píxel $\text{imagen}[0][j]$ y su último píxel $\text{imagen}[n-1][j]$:

1. Si $\text{imagen}[0][j] == 1$ y $\text{imagen}[n-1][j] == 1 \implies$ **Columna mineral** (todos son 1, longitud = n).
2. Si $\text{imagen}[0][j] == 0$ y $\text{imagen}[n-1][j] == 0 \implies$ **Nada** (todos son 0, longitud = 0).
3. Si $\text{imagen}[0][j] == 1$ y $\text{imagen}[n-1][j] == 0 \implies$ **Estalactita** (los 1 están arriba, los 0 abajo).
4. Si $\text{imagen}[0][j] == 0$ y $\text{imagen}[n-1][j] == 1 \implies$ **Estalagmita** (los 0 están arriba, los 1 abajo).

Búsqueda de la Longitud en $\mathcal{O}(\log n)$

Una vez identificada la categoría, podemos encontrar el punto de transición exacto entre 1 y 0 aplicando **Búsqueda Binaria** (Divide y Vencerás) sobre las filas de la columna:

- **Para una Estalactita:** Buscamos el **primer** 0 de la columna. El índice de este primer 0 (0-indexado) coincidirá exactamente con la cantidad de 1s (longitud de la estalactita).

- **Para una Estalagmita:** Buscamos el **primer 1** de la columna. La longitud de la estalagmita será $n - \text{índice del primer 1}$.

2. Algoritmo y Pseudocódigo

Búsqueda Binaria para Estalactita (Buscar primer 0)

Busca de forma binaria la transición de 1 a 0 en la columna *col* entre las filas *inicio* y *fin*:

```

Función BuscarPrimerCero(imagen, col, inicio, fin):
    Si inicio > fin:
        Retornar inicio // Punto de transición

    medio = (inicio + fin) / 2

    Si imagen[medio][col] == 0:
        // El primer cero está en 'medio' o a su izquierda
        Retornar BuscarPrimerCero(imagen, col, inicio, medio - 1)
    Sino:
        // 'medio' sigue siendo un 1, el primer cero está a la derecha
        Retornar BuscarPrimerCero(imagen, col, medio + 1, fin)

```

Búsqueda Binaria para Estalagmita (Buscar primer 1)

Busca de forma binaria la transición de 0 a 1 en la columna *col* entre las filas *inicio* y *fin*:

```

Función BuscarPrimerUno(imagen, col, inicio, fin):
    Si inicio > fin:
        Retornar inicio // Punto de transición

    medio = (inicio + fin) / 2

    Si imagen[medio][col] == 1:
        // El primer uno está en 'medio' o a su izquierda
        Retornar BuscarPrimerUno(imagen, col, inicio, medio - 1)
    Sino:
        // 'medio' sigue siendo un 0, el primer uno está a la derecha
        Retornar BuscarPrimerUno(imagen, col, medio + 1, fin)

```

Algoritmo Principal

```

Función CalcularMaximoElemento(imagen, n, m):
    max_estalactita = 0
    max_estalagmita = 0

    Para j desde 0 hasta m - 1:
        primer_pixel = imagen[0][j]
        ultimo_pixel = imagen[n-1][j]

        // 1. Identificar categoría y resolver en O(log n)
        Si (primer_pixel == 1 y ultimo_pixel == 0):

```

```

// Caso Estalactita
longitud = BuscarPrimerCero(imagen, j, 0, n - 1)
max_estalactita = Maximo(max_estalactita, longitud)

Sino si (primer_pixel == 0 y ultimo_pixel == 1):
// Caso Estalagmita
primer_uno = BuscarPrimerUno(imagen, j, 0, n - 1)
longitud = n - primer_uno
max_estalagmita = Maximo(max_estalagmita, longitud)

Retornar Maximo(max_estalactita, max_estalagmita)

```

3. Análisis de Complejidad

- **Complejidad Temporal:**
 - **Análisis de una columna:** Determinar la categoría toma $\mathcal{O}(1)$ tiempo. La búsqueda binaria sobre una columna de altura n divide el espacio de búsqueda a la mitad en cada paso, costando exactamente $\mathcal{O}(\log n)$ comparaciones.
 - **Coste total:** Al repetir este proceso para las m columnas, el coste temporal total es estrictamente de $\mathcal{O}(m \log n)$, cumpliendo con la restricción del enunciado.
- **Complejidad Espacial:**
 - $\mathcal{O}(1)$ (espacio auxiliar constante) si implementamos las búsquedas binarias de forma iterativa, o bien $\mathcal{O}(\log n)$ de espacio en la pila de llamadas del sistema si usamos las funciones recursivas propuestas.

Ejercicio 7: Mediana de dos vectores ordenados (Divide y Vencerás)

Enunciado

Dados dos vectores de enteros $X[1 \dots n]$ e $Y[1 \dots n]$, ordenados de forma creciente, diseña un algoritmo que calcule la mediana de los $2n$ elementos en tiempo $\mathcal{O}(\log n)$ y sin utilizar vectores adicionales.

1. Concepto y Estrategia de Divide y Vencerás

La mediana de un conjunto de $2n$ elementos ordenados (número par de elementos) es el **elemento inferior de la mediana** (el elemento en la posición n de los $2n$ elementos ordenados, usando indexación de 1 a $2n$).

En lugar de mezclar los dos vectores en un único vector de tamaño $2n$ (lo cual costaría $\mathcal{O}(n)$ en tiempo y espacio), aplicamos una estrategia de **Divide y Vencerás**:

1. Calculamos el punto medio de los subvectores actuales, $m = \lfloor n/2 \rfloor$.
2. Comparamos los elementos medianos de ambos subvectores: $X[x_i + m - 1]$ y $Y[y_i + m - 1]$.
3. **Descarte de mitades**:
 - Si $X[x_i + m - 1] \leq Y[y_i + m - 1]$: Significa que la mediana global no puede estar en la primera mitad de X ni en la segunda mitad de Y . Descartamos estas zonas y repetimos la búsqueda sobre la segunda mitad de X y la primera mitad de Y .
 - Si $X[x_i + m - 1] > Y[y_i + m - 1]$: Descartamos la segunda mitad de X y la primera mitad de Y .

Cada paso recursivo reduce el tamaño del problema a la mitad, garantizando un coste de $\mathcal{O}(\log n)$.

2. Explicación de los Casos Base (Según la Pizarra)

Cuando el tamaño del intervalo actual n llega a ser muy pequeño ($n = 1$ y $n = 2$), se aplican las siguientes reglas directas para evitar recursión infinita o desbordamiento de índices:

- **Caso $n = 1$** : Tenemos solo dos elementos: $X[x_i]$ y $Y[y_i]$. La mediana (el menor de los dos) es sencillamente:

$$\min(X[x_i], Y[y_i])$$

- **Caso $n = 2$** : Tenemos cuatro elementos en juego: $\{X[x_i], X[x_s]\}$ y $\{Y[y_i], Y[y_s]\}$. Queremos encontrar el segundo elemento más pequeño de los cuatro:

- **Caso 1** ($X[x_s] < Y[y_i]$): Al estar ordenados, el orden total es $X[x_i], X[x_s], Y[y_i], Y[y_s]$. El segundo elemento es $X[x_s]$.
- **Caso 2** ($Y[y_s] < X[x_i]$): Por simetría, el orden total es $Y[y_i], Y[y_s], X[x_i], X[x_s]$. El segundo elemento es $Y[y_s]$.
- **Caso 3 (Por defecto)**: Si hay solapamiento entre los intervalos, el segundo elemento más pequeño será el máximo de los dos elementos iniciales: $\max(X[x_i], Y[y_i])$.

3. Algoritmo Completo en Pseudocódigo

```
// Función recursiva principal
Función Mediana2V(X, Y, xi, xs, yi, ys):
    n = xs - xi + 1 // Tamaño del subproblema actual

    // CASO BASE 1: Solo queda un elemento en cada subvector
    Si (n == 1):
        Retornar Minimo(X[xi], Y[yi])

    // CASO BASE 2: Quedan dos elementos en cada subvector
    Si (n == 2):
        Si (X[xs] < Y[yi]):
            Retornar X[xs] // Caso 1 (X totalmente a la izquierda de Y)
        Si (Y[ys] < X[xi]):
            Retornar Y[ys] // Caso 2 (Y totalmente a la izquierda de X)
        Retornar Maximo(X[xi], Y[yi]) // Caso 3 (Intervalos solapados)

    // DIVIDE Y VENCERÁS (Reducción del tamaño a la mitad)
    m = parte_entera_inferior(n / 2)

    // Comparación de los elementos centrales de los subvectores
    Si (X[xi + m - 1] <= Y[yi + m - 1]):
        // Descartamos la primera mitad de X y la segunda mitad de Y
        Retornar Mediana2V(X, Y, xi + m, xs, yi, ys - m)
    Sino:
        // Descartamos la segunda mitad de X y la primera mitad de Y
        Retornar Mediana2V(X, Y, xi, xs - m, yi + m, ys)

// Función envoltura (Llamada inicial)
Función CalcularMediana(X, Y, n):
    Si (n == 0):
        Retornar "Error: Vectores vacíos"
    Retornar Mediana2V(X, Y, 1, n, 1, n)
```

4. Análisis de Complejidad

- **Complejidad Temporal:** $\mathcal{O}(\log n)$. En cada paso recursivo calculamos $m = \lfloor n/2 \rfloor$ y descartamos exactamente m elementos de un vector y m del otro. La ecuación de recurrencia del algoritmo es:

$$T(n) = T(n/2) + \mathcal{O}(1)$$

Ejercicio 8: Matriz de Rastreo (Divide y Vencerás)

Enunciado

Una matriz de rastreo es una matriz cuadrada de $n \times n$ números enteros (sin repeticiones), siendo n una potencia de 2, y en la que se cumple que $a_1 < b_1 < a_2 < b_2 < a_3 < b_3 < a_4 < b_4$ y donde además las cuatro sub-matrices componentes son también matrices de rastreo.

Diseñar un algoritmo de tipo Divide y Vencerás que determine si la matriz dada es o no de rastreo. Analizar su coste en tiempo.

1. Estrategia de Divide y Vencerás

Por la definición del problema, una matriz M de tamaño $n \times n$ es una **Matriz de Rastreo** si y solo si cumple las siguientes tres condiciones:

1. **Caso Base:** Si $n = 1$, una matriz de 1×1 siempre es una matriz de rastreo válida (un único elemento no puede tener repeticiones ni violar desigualdades).
2. **Propiedad de las Esquinas (Paso de Combinación):** Los elementos de las esquinas clave de las cuatro submatrices de tamaño $(n/2) \times (n/2)$ deben cumplir la cadena de desigualdades estrictas:

$$a_1 < b_1 < a_2 < b_2 < a_3 < b_3 < a_4 < b_4$$

Donde identificamos los índices en la matriz M (usando indexación de 1 a n):

- $a_1 = M[1][1]$
 - $b_1 = M[n/2][n/2]$
 - $a_2 = M[1][n/2 + 1]$
 - $b_2 = M[n/2][n]$
 - $a_3 = M[n/2 + 1][1]$
 - $b_3 = M[n][n/2]$
 - $a_4 = M[n/2 + 1][n/2 + 1]$
 - $b_4 = M[n][n]$
3. **Condición Recursiva (Divide y Vencerás):** Las cuatro submatrices componentes de tamaño $(n/2) \times (n/2)$ deben ser, a su vez, matrices de rastreo válidas.

2. Algoritmo en Pseudocódigo

Para poder subdividir la matriz sin realizar copias físicas (lo cual degradaría la eficiencia espacial y temporal), el algoritmo trabaja con coordenadas límites de la submatriz actual:

(fila_inicio, fila_fin) y (col_inicio, col_fin) .

```
// Función recursiva principal
Función EsMatrizRastreo(M, f_ini, f_fin, c_ini, c_fin):
    n = f_fin - f_ini + 1 // Tamaño de la submatriz actual

    // CASO BASE: Matriz de 1x1
    Si (n == 1):
        Retornar Verdadero

    // 1. Identificar las coordenadas divisorias (mitad)
    mitad = n / 2
    f_med = f_ini + mitad - 1
    c_med = c_ini + mitad - 1

    // 2. Extraer los elementos clave de las esquinas (a_i, b_i)
    a1 = M[f_ini][c_ini]
    b1 = M[f_med][c_med]

    a2 = M[f_ini][c_med + 1]
    b2 = M[f_med][c_fin]

    a3 = M[f_med + 1][c_ini]
    b3 = M[f_fin][c_med]

    a4 = M[f_med + 1][c_med + 1]
    b4 = M[f_fin][c_fin]

    // 3. Comprobar la restricción de desigualdad del nivel actual
    Si NO (a1 < b1 y b1 < a2 y a2 < b2 y b2 < a3 y a3 < b3 y b3 < a4 y a4 < b4):
        Retornar Falso

    // 4. DIVIDIR: Comprobar recursivamente las 4 submatrices componentes
    sub1 = EsMatrizRastreo(M, f_ini, f_med, c_ini, c_med) // Sup-Izq
    sub2 = EsMatrizRastreo(M, f_ini, f_med, c_med + 1, c_fin) // Sup-Der
    sub3 = EsMatrizRastreo(M, f_med + 1, f_fin, c_ini, c_med) // Inf-Izq
    sub4 = EsMatrizRastreo(M, f_med + 1, f_fin, c_med + 1, c_fin) // Inf-Der

    // Vencer: Solo es de rastreo si las cuatro submatrices lo son
    Retornar (sub1 y sub2 y sub3 y sub4)

// Función envoltura (Llamada inicial)
Función ValidarRastreo(M, n):
    Si (n == 0 o NO es_potencia_de_2(n)):
        Retornar Falso
    Retornar EsMatrizRastreo(M, 1, n, 1, n)
```

3. Análisis de Coste Temporal

Para analizar la eficiencia temporal, planteamos la ecuación de recurrencia del algoritmo:

- En cada paso no base de tamaño n , el algoritmo realiza una cantidad constante de operaciones $\mathcal{O}(1)$ para calcular los índices intermedios, acceder a los 8 elementos de las esquinas y comprobar las desigualdades.

- Posteriormente, divide el problema realizando **4 llamadas recursivas** sobre problemas de tamaño mitad ($n/2$).

La ecuación de recurrencia es:

$$T(n) = 4T\left(\frac{n}{2}\right) + \mathcal{O}(1)$$

Aplicación del Teorema Máster

El Teorema Máster se define para ecuaciones de la forma $T(n) = aT(n/b) + \Theta(n^d)$:

- $a = 4$ (número de subproblemas)
- $b = 2$ (factor de reducción de tamaño)
- $d = 0$ (ya que el trabajo de combinación/división es constante, $\mathcal{O}(1) = \Theta(n^0)$)

Comparamos los valores:

$$\log_b(a) = \log_2(4) = 2$$

Dado que $\log_b(a) > d$ ($2 > 0$), nos encontramos en el caso donde el coste está dominado por las llamadas recursivas de las hojas del árbol:

$$T(n) \in \Theta(n^{\log_b(a)}) = \Theta(n^2)$$

Ejercicio 9: Multiplicación Marciana (Algoritmo de tipo Karatsuba)

Enunciado

Has encontrado una colonia de extraterrestres que viven en Marte y has estado aprendiendo más sobre su aritmética. En particular, has descubierto que usan el mismo sistema numérico en base 10, suma y resta igual que nosotros, pero su sistema de multiplicación es completamente diferente. El símbolo que usan es $\star$ y tienen una tabla de multiplicar de 1 dígito que usan en la escuela primaria. Aquí hay unos ejemplos:

Después de un rato, te has enterado que su multiplicación corresponde a lo siguiente en la aritmética humana:

$$x \star y = x^2 + xy + y^2$$

Quieres explicarles una forma eficiente de calcular el producto marciano de dos números de n dígitos, sin hacer referencia ni explicar ninguna multiplicación humana (incluso implícitamente como una suma). Por ejemplo, no puedes decirles que calculen x^2 , xy y y^2 por separado, ya que no entienden qué significan estos términos.

Encuentra un algoritmo para calcular el producto marciano de dos números de n dígitos que se ejecute en tiempo $\mathcal{O}(n^{\log_2 3})$. Puedes suponer que n es una potencia de 2, y que los ordenadores de los marcianos pueden hacer operaciones de shift, suma y resta exactamente como los ordenadores humanos. También puedes suponer que los ordenadores de los marcianos tienen fácil acceso a la tabla de multiplicar de 1 dígito.

1. El Conflicto Conceptual (Evitar la Multiplicación Humana)

El punto delicado del ejercicio es el siguiente: el enunciado **prohíbe estrictamente** realizar multiplicaciones humanas estándar (como calcular individualmente x^2 , xy o y^2 y luego sumarlos), ya que los marcianos no comprenden el concepto de multiplicación humana.

Para resolverlo, debemos diseñar una estrategia de **Divide y Vencerás** (similar al algoritmo de Karatsuba) que combine las descomposiciones de los números utilizando exclusivamente:

- Sumas y restas de números.
- Desplazamientos de dígitos (multiplicar por potencias de 10, que son simples *shifts* a nivel de máquina).
- Llamadas recursivas al propio operador marciano $\star$.

2. Desarrollo Algebraico de la Solución

Sea n una potencia de 2. Dividimos los dos números de n dígitos, x e y , en sus mitades más significativas y menos significativas de tamaño $m = n/2$:

$$x = x_1 \cdot 10^m + x_0$$

$$y = y_1 \cdot 10^m + y_0$$

Para simplificar el álgebra, denotemos la base de desplazamiento como $B = 10^m$:

$$x = x_1 B + x_0$$

$$y = y_1 B + y_0$$

Queremos calcular el producto marciano $x \star y = x^2 + xy + y^2$. Expandimos algebraicamente cada término:

1. El término cuadrático x^2 :

$$x^2 = (x_1 B + x_0)^2 = x_1^2 B^2 + 2x_1 x_0 B + x_0^2$$

2. El término cruzado xy :

$$xy = (x_1 B + x_0)(y_1 B + y_0) = x_1 y_1 B^2 + (x_1 y_0 + x_0 y_1) B + x_0 y_0$$

3. El término cuadrático y^2 :

$$y^2 = (y_1 B + y_0)^2 = y_1^2 B^2 + 2y_1 y_0 B + y_0^2$$

Sumamos los tres componentes para obtener el producto marciano total:

$$x \star y = (x_1^2 + x_1 y_1 + y_1^2) B^2 + (2x_1 x_0 + x_1 y_0 + x_0 y_1 + 2y_1 y_0) B + (x_0^2 + x_0 y_0 + y_0^2)$$

3. Reducción de 4 a 3 Multiplicaciones (Truco de Karatsuba)

Analizamos los tres coeficientes de nuestra ecuación agrupada:

- **Coefficiente de B^2 :** Es exactamente el producto marciano de las mitades superiores:

$$P = x_1 \star y_1 = x_1^2 + x_1 y_1 + y_1^2$$

- **Coefficiente independiente (término constante):** Es el producto marciano de las mitades inferiores:

$$Q = x_0 \star y_0 = x_0^2 + x_0 y_0 + y_0^2$$

- **Coefficiente central (multiplicado por B):**

$$\text{medio} = 2x_1x_0 + x_1y_0 + x_0y_1 + 2y_1y_0$$

Para calcular el término **medio** sin realizar multiplicaciones humanas, calculamos recursivamente el producto marciano de las sumas de las mitades, al que llamaremos R :

$$R = (x_1 + x_0) \star (y_1 + y_0)$$

Si expandimos la definición del producto marciano para R :

$$R = (x_1 + x_0)^2 + (x_1 + x_0)(y_1 + y_0) + (y_1 + y_0)^2$$

$$R = (x_1^2 + 2x_1x_0 + x_0^2) + (x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0) + (y_1^2 + 2y_1y_0 + y_0^2)$$

Reordenando los términos agrupados:

$$R = \underbrace{(x_1^2 + x_1y_1 + y_1^2)}_P + \underbrace{(x_0^2 + x_0y_0 + y_0^2)}_Q + \underbrace{(2x_1x_0 + x_1y_0 + x_0y_1 + 2y_1y_0)}_{\text{medio}}$$

$$R = P + Q + \text{medio}$$

Despejando el término central que necesitamos:

$$\text{medio} = R - P - Q$$

Expresión Final del Algoritmo

Podemos reescribir la operación completa usando únicamente sumas, restas, desplazamientos (*shifts* por potencias de 10) y 3 llamadas recursivas al producto marciano:

$$x \star y = P \cdot 10^n + (R - P - Q) \cdot 10^{n/2} + Q$$

4. Algoritmo en Pseudocódigo

```
// Función recursiva para la multiplicación marciana
Función ProductoMarciano(x, y, n):
    // CASO BASE: Números de 1 dígito
    Si (n == 1):
        // Consultar directamente la tabla de multiplicar marciana de primaria
        Retornar TablaMarciana[x][y]

    m = n / 2
    B = 10^m // Factor de desplazamiento
```



```

// División de los números en mitades (sin multiplicaciones)
x1 = parte_entera(x / B)
x0 = x % B

y1 = parte_entera(y / B)
y0 = y % B

// DIVIDIR: Las 3 llamadas recursivas
P = ProductoMarciano(x1, y1, m)
Q = ProductoMarciano(x0, y0, m)
R = ProductoMarciano(x1 + x0, y1 + y0, m)

// COMBINAR (Vencer): Sumas, restas y desplazamientos de base
// P * B * B equivale a desplazar P por n posiciones (P * 10^n)
// (R - P - Q) * B equivale a desplazar por n/2 posiciones
resultado = (P * B * B) + ((R - P - Q) * B) + Q

Retornar resultado

```

5. Análisis de Complejidad Temporal

Planteamos la ecuación de recurrencia del algoritmo:

- Hacemos exactamente **3 llamadas recursivas** de tamaño mitad ($n/2$).
- Las operaciones de división de números, sumas, restas y desplazamientos de dígitos (*multiplicar por potencias de 10*) se realizan en tiempo lineal $\mathcal{O}(n)$.

La recurrencia es:

$$T(n) = 3T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Aplicación del Teorema Máster

Comparamos con la forma general $T(n) = aT(n/b) + \Theta(n^d)$:

- $a = 3$
- $b = 2$
- $d = 1$ (el trabajo de combinación y partición es lineal, $\Theta(n^1)$)

Calculamos el valor crítico:

$$\log_b(a) = \log_2(3) \approx 1.585$$

Dado que $\log_b(a) > d$ ($1.585 > 1$), aplicamos el caso correspondiente del Teorema Máster:

$$T(n) \in \Theta(n^{\log_b(a)}) = \Theta(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$$

Resolución: Conteo Eficiente de Inversiones en una Lista

Este documento presenta la resolución detallada del problema de calcular el número de inversiones en una lista $L[1 \dots n]$ utilizando la estrategia de **Divide y Vencerás** mediante una adaptación directa del algoritmo de ordenación **MergeSort**.

a) Máximo Número de Inversiones en una Lista de Longitud n

Respuesta

El máximo número de inversiones que podemos encontrar en una lista de longitud n es:

$$\frac{n(n-1)}{2} = \binom{n}{2}$$

Justificación

Una inversión se define como un par de índices (i, j) tales que $i < j$ y $L[i] > L[j]$.

1. **Número total de pares:** En cualquier lista de longitud n , el número total de pares de índices posibles (i, j) con $i < j$ corresponde a las combinaciones de elegir 2 elementos de entre un conjunto de n , lo cual equivale a:

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

2. **Caso de máxima inversión:** El número máximo de inversiones ocurre cuando **todos** los pares posibles de la lista forman una inversión. Esto sucede si y solo si la lista está ordenada de manera **estrictamente decreciente** (por ejemplo, $L = [n, n-1, \dots, 2, 1]$). En este escenario, para cualquier par de índices $i < j$, siempre se cumplirá que $L[i] > L[j]$.

Dado que no pueden existir más parejas de índices que las $\frac{n(n-1)}{2}$ totales, este valor representa el límite superior absoluto de inversiones posibles.

b) Descripción del Algoritmo Eficiente (Basado en MergeSort)

Para diseñar un algoritmo de coste $O(n \log n)$, adaptamos el funcionamiento de **MergeSort** (Ordenación por Mezcla). La gran ventaja de este enfoque es que, al fusionar dos sublistas ya ordenadas, podemos detectar y contabilizar múltiples inversiones de golpe (en bloque) sin necesidad de comparar cada elemento de uno en uno.

Idea de la Adaptación

1. **Dividir (Divide):** Dividimos la lista $L[izq \dots der]$ por la mitad calculando el punto medio mid .

2. **Conquistar (Conquer):** De forma recursiva, ordenamos y contamos las inversiones de la mitad izquierda $L[izq \dots mid]$ y de la mitad derecha $L[mid + 1 \dots der]$.
3. **Combinar (Merge):** Fusionamos (mezclamos) las dos mitades ya ordenadas en una estructura temporal y, durante este proceso, contamos las **inversiones cruzadas** (elementos de la mitad izquierda que son estrictamente mayores que elementos de la mitad derecha).

Lógica de Conteo en el Proceso de Mezcla (Merge)

Supongamos que estamos mezclando la mitad izquierda $L[izq \dots mid]$ y la mitad derecha $L[mid + 1 \dots der]$, ambas ya ordenadas. Usamos dos punteros para recorrerlas: i para la izquierda y j para la derecha.

- Si $L[i] \leq L[j]$: El elemento $L[i]$ se coloca en su posición correcta en el vector temporal. No hay inversión. Avanzamos el puntero i .
- Si $L[i] > L[j]$: Se ha detectado una inversión. Como la mitad izquierda ya está ordenada ascendentemente, **todos** los elementos que queden en la mitad izquierda (desde la posición actual i hasta mid) son garantizadamente mayores que $L[j]$.
 - Por tanto, el elemento $L[j]$ forma exactamente $(mid - i + 1)$ inversiones con la sublista izquierda.
 - Sumamos este valor directamente al contador de inversiones y avanzamos el puntero j .

c) Algoritmo en Pseudocódigo

A continuación se detalla el pseudocódigo del algoritmo estructurado con índices base 1:

```

Funcion MergeYContar(L, izq, mid, der)
Inicio
  temp <- Vector de tamaño (der - izq + 1)
  i <- izq      // Puntero para la mitad izquierda
  j <- mid + 1  // Puntero para la mitad derecha
  k <- 1        // Puntero para el vector temporal
  inversiones_cruzadas <- 0

  Mientras i <= mid y j <= der Hacer
    Si L[i] <= L[j] Entonces
      temp[k] <- L[i]
      i <- i + 1
    Sino
      temp[k] <- L[j]
      // Sumamos todas las inversiones que produce L[j] con el bloque izqui
      inversiones_cruzadas <- inversiones_cruzadas + (mid - i + 1)
      j <- j + 1
    FinSi
    k <- k + 1
  FinMientras

  // Copiar elementos restantes de la mitad izquierda (si quedan)
  Mientras i <= mid Hacer
    temp[k] <- L[i]

```

```

        i <- i + 1
        k <- k + 1
FinMientras

// Copiar elementos restantes de la mitad derecha (si quedan)
Mientras j <= der Hacer
    temp[k] <- L[j]
    j <- j + 1
    k <- k + 1
FinMientras

// Volcar el contenido del arreglo temporal de vuelta a la lista original L
Para p <- 0 Hasta (der - izq) Con Paso 1 Hacer
    L[izq + p] <- temp[p + 1]
FinPara

Retornar inversiones_cruzadas
FinFuncion

Funcion MergeSortInversiones(L, izq, der)
Inicio
    Si izq >= der Entonces
        Retornar 0 // Caso base: un solo elemento no tiene inversiones
    FinSi

    mid <- ParteEntera((izq + der) / 2)

    // Contar las inversiones internas de cada mitad y las cruzadas al mezclar
    inv_izq <- MergeSortInversiones(L, izq, mid)
    inv_der <- MergeSortInversiones(L, mid + 1, der)
    inv_cruzadas <- MergeYContar(L, izq, mid, der)

    Retornar inv_izq + inv_der + inv_cruzadas
FinFuncion

Funcion ContarInversiones(L, n)
Inicio
    // Llamada inicial para procesar el vector completo L[1..n]
    Retornar MergeSortInversiones(L, 1, n)
FinFuncion

```

d) Análisis del Coste Asintótico en Tiempo

1. **Ecuación de Recurrencia:** El tiempo de ejecución $T(n)$ para procesar un arreglo de tamaño n se descompone en resolver dos subproblemas de tamaño $n/2$ de manera recursiva más una fase de combinación lineal $O(n)$ correspondiente a la fusión de subarreglos:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) = T(n) \in O(n \log n)$$